

第 27 讲核心概念速查卡

PyTorch Tensor

一句话：`torch.Tensor` 是“带 `dtype`、`device`、`autograd` 等深度学习运行信息的多维数字容器”。

1. Tensor 五个核心属性

属性	含义
<code>values</code>	Tensor 里面的实际数字
<code>shape</code>	每个轴有多长，例如 (2, 3, 4)
<code>ndim</code>	有几个轴，例如 (2, 3, 4) 的 <code>ndim</code> 为 3
<code>dtype</code>	数字的存储类型，例如 <code>float32</code> 、 <code>float64</code> 、 <code>int64</code>
<code>device</code>	Tensor 当前位于 CPU 还是 GPU
<code>requires_grad</code>	是否需要记录用于自动求导的计算依赖

2. NumPy 与 PyTorch 对应

概念	NumPy	PyTorch
多维数据	<code>ndarray</code>	<code>Tensor</code>
Shape	<code>x.shape</code>	<code>x.shape</code>
维数	<code>x.ndim</code>	<code>x.ndim</code>
逐元素乘法	<code>A * B</code>	<code>A * B</code>
矩阵乘法	<code>A @ B</code>	<code>A @ B</code>
改变 Shape	<code>reshape</code>	<code>reshape</code>

3. 索引与 Shape

如果 `X.shape = (2, 3)`:

代码	Shape	说明
<code>X[0]</code>	(3,)	整数索引移除该维
<code>X[0:1]</code>	(1, 3)	切片保留该维
<code>X[:, 1]</code>	(2,)	列维被移除
<code>X[:, 1:2]</code>	(2, 1)	列维被保留

4. * 与 @

$A * B$ = 逐元素乘法

$$A@W = \text{矩阵乘法}$$

例如：

$$(2, 3)@(3, 4) \rightarrow (2, 4)$$

5. dtype

常见：

- `torch.float32`: 常见训练浮点类型
- `torch.float64`: 更高精度、更大内存开销
- `torch.float16` / `bfloat16`: 大模型常见低精度浮点
- `torch.int64`: Token ID、类别标签常见

NumPy 的 Python 浮点数组常默认 `float64`；PyTorch 的 Python 浮点 Tensor 常默认 `float32`。科研代码里不要凭感觉猜 `dtype`，要打印确认。

6. device

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
x = x.to(device)
```

同一运算中的 Tensor 通常必须位于兼容 device。

7. NumPy 转 Tensor

```
t_shared = torch.from_numpy(a) # 常见 CPU 场景共享内存
t_copy   = torch.tensor(a)     # 通常复制
```

Tensor 转 NumPy：

```
x.detach().cpu().numpy()
```

8. requires_grad

```
x = torch.tensor([2.0], requires_grad=True)
y = x * 3 + 1
```

含义：PyTorch 开始记录梯度依赖。

`requires_grad=True` 不等于已经算出梯度。真正把梯度写入 `x.grad`，要等后面执行 `backward`。

9. LLM Shape 连接

$$X : (B, T, D), \quad W : (D, O)$$

$$X@W : (B, T, O)$$

Transformer 的 $Q = XW_Q$ 、 $K = XW_K$ 、 $V = XW_V$ 继续使用同一套规则。

10. 本讲最小检查清单

看到 Tensor 时先问：

1. Shape 是多少？每个轴代表什么？
2. dtype 是什么？
3. device 在哪里？
4. requires_grad 是 True 还是 False？
5. 当前运算是逐元素还是矩阵运算？